Anabela Petrushevikj D209 Data Mining I Task IResearch question: What factors influence the likelihood of a patient being readmitted to the hospital within 30 days of discharge?

In [1]:
```
#Importing libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import zscore
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, accuracy_score, ConfusionMatrixDisplay, roc_auc_sc
```

In [2]:
```
#Loading the dataset
df = pd.read_csv('medical_clean_D209_I.csv')
```

In [3]:
```
#Information about the dataset
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 50 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   CaseOrder           10000 non-null  int64
 1   Customer_id         10000 non-null  object
 2   Interaction         10000 non-null  object
 3   UID                 10000 non-null  object
 4   City                10000 non-null  object
 5   State               10000 non-null  object
 6   County              10000 non-null  object
 7   Zip                 10000 non-null  int64
 8   Lat                 10000 non-null  float64
 9   Lng                 10000 non-null  float64
 10  Population          10000 non-null  int64
 11  Area                10000 non-null  object
 12  TimeZone            10000 non-null  object
 13  Job                 10000 non-null  object
 14  Children            10000 non-null  int64
 15  Age                 10000 non-null  int64
 16  Income              10000 non-null  float64
 17  Marital             10000 non-null  object
 18  Gender              10000 non-null  object
 19  ReAdmis             10000 non-null  object
 20  VitD_levels         10000 non-null  float64
 21  Doc_visits          10000 non-null  int64
 22  Full_meals_eaten    10000 non-null  int64
 23  vitD_supp           10000 non-null  int64
 24  Soft_drink          10000 non-null  object
 25  Initial_admin       10000 non-null  object
 26  HighBlood           10000 non-null  object
 27  Stroke              10000 non-null  object
 28  Complication_risk   10000 non-null  object
 29  Overweight          10000 non-null  object
 30  Arthritis           10000 non-null  object
 31  Diabetes            10000 non-null  object
 32  Hyperlipidemia      10000 non-null  object
 33  BackPain            10000 non-null  object
 34  Anxiety             10000 non-null  object
 35  Allergic_rhinitis   10000 non-null  object
 36  Reflux_esophagitis  10000 non-null  object
 37  Asthma              10000 non-null  object
 38  Services            10000 non-null  object
 39  Initial_days        10000 non-null  float64
 40  TotalCharge         10000 non-null  float64
 41  Additional_charges  10000 non-null  float64
 42  Item1               10000 non-null  int64
 43  Item2               10000 non-null  int64
 44  Item3               10000 non-null  int64
 45  Item4               10000 non-null  int64
 46  Item5               10000 non-null  int64
 47  Item6               10000 non-null  int64
 48  Item7               10000 non-null  int64
 49  Item8               10000 non-null  int64
dtypes: float64(7), int64(16), object(27)
memory usage: 3.8+ MB
```

In [4]:
```
#Remove irrelevant columns
```

```
df.drop(columns=['CaseOrder', 'Customer_id', 'Interaction', 'UID', 'Lat', 'Lng', 'TimeZone', 'It
```

In [5]:
```
#Checking for duplicate values
print(df.duplicated())
```

```
0       False
1       False
2       False
3       False
4       False
        ...
9995    False
9996    False
9997    False
9998    False
9999    False
Length: 10000, dtype: bool
```

In [6]:
```
#Checking for missing values
print(df.isnull().sum())
```

```
City                  0
State                 0
County                0
Zip                   0
Population            0
Area                  0
Job                   0
Children              0
Age                   0
Income                0
Marital               0
Gender                0
ReAdmis               0
VitD_levels           0
Doc_visits            0
Full_meals_eaten      0
vitD_supp             0
Soft_drink            0
Initial_admin         0
HighBlood             0
Stroke                0
Complication_risk     0
Overweight            0
Arthritis             0
Diabetes              0
Hyperlipidemia        0
BackPain              0
Anxiety               0
Allergic_rhinitis     0
Reflux_esophagitis    0
Asthma                0
Services              0
Initial_days          0
TotalCharge           0
Additional_charges    0
dtype: int64
```

In [7]:
```
#Encode categorical variables using LabelEncoder
label_encoder = LabelEncoder()
```

```
categorical_cols = df.select_dtypes(include=['object']).columns

for col in categorical_cols:
    df[col] = label_encoder.fit_transform(df[col])
```

In [8]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 35 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   City                10000 non-null  int32
 1   State               10000 non-null  int32
 2   County              10000 non-null  int32
 3   Zip                 10000 non-null  int64
 4   Population          10000 non-null  int64
 5   Area                10000 non-null  int32
 6   Job                 10000 non-null  int32
 7   Children            10000 non-null  int64
 8   Age                 10000 non-null  int64
 9   Income              10000 non-null  float64
 10  Marital             10000 non-null  int32
 11  Gender              10000 non-null  int32
 12  ReAdmis             10000 non-null  int32
 13  VitD_levels         10000 non-null  float64
 14  Doc_visits          10000 non-null  int64
 15  Full_meals_eaten    10000 non-null  int64
 16  vitD_supp           10000 non-null  int64
 17  Soft_drink          10000 non-null  int32
 18  Initial_admin       10000 non-null  int32
 19  HighBlood           10000 non-null  int32
 20  Stroke              10000 non-null  int32
 21  Complication_risk   10000 non-null  int32
 22  Overweight          10000 non-null  int32
 23  Arthritis           10000 non-null  int32
 24  Diabetes            10000 non-null  int32
 25  Hyperlipidemia      10000 non-null  int32
 26  BackPain            10000 non-null  int32
 27  Anxiety             10000 non-null  int32
 28  Allergic_rhinitis   10000 non-null  int32
 29  Reflux_esophagitis  10000 non-null  int32
 30  Asthma              10000 non-null  int32
 31  Services            10000 non-null  int32
 32  Initial_days        10000 non-null  float64
 33  TotalCharge         10000 non-null  float64
 34  Additional_charges  10000 non-null  float64
dtypes: float64(5), int32(23), int64(7)
memory usage: 1.8 MB
```

In [9]:
```
#Identify outliers based on Z-scores
z_scores = np.abs(zscore(df))
z_score_outliers = (z_scores > 3).sum(axis=0)
print("Outliers based on Z-score method:")
print(z_score_outliers)
```

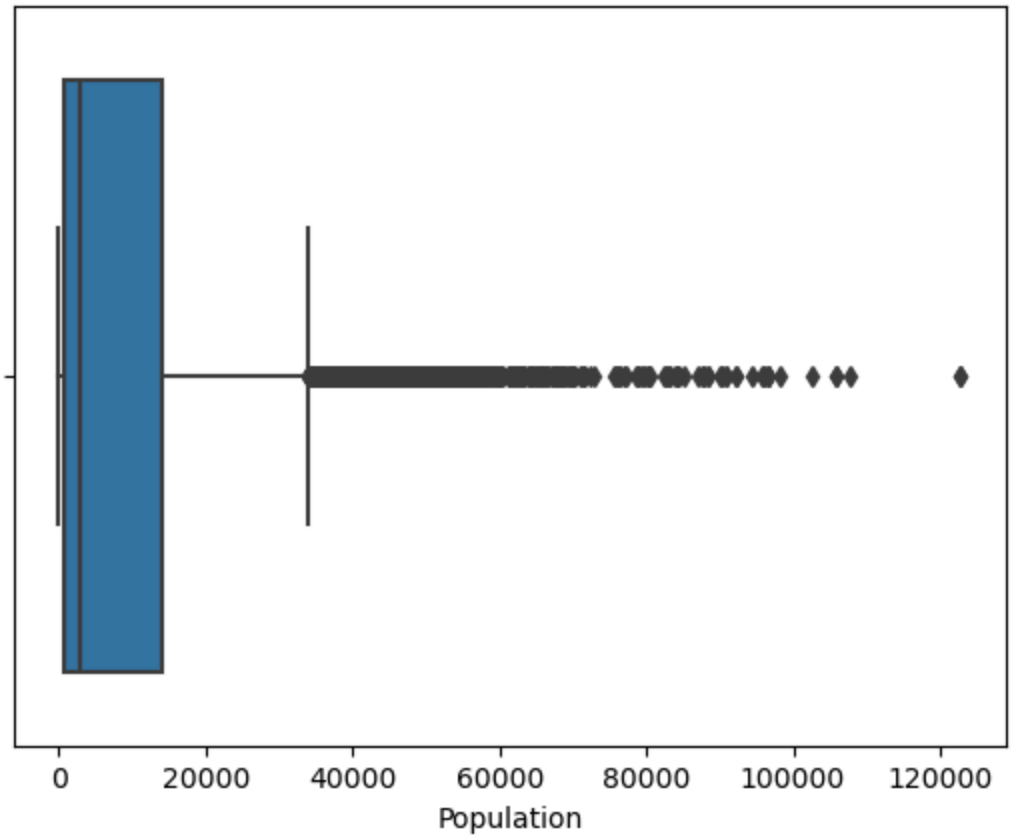
```
Outliers based on Z-score method:
City                    0
State                   0
County                  0
Zip                     0
Population            218
Area                    0
Job                     0
Children              202
Age                     0
Income                143
Marital                 0
Gender                  0
ReAdmis                 0
VitD_levels            24
Doc_visits              8
Full_meals_eaten       33
vitD_supp              70
Soft_drink              0
Initial_admin           0
HighBlood               0
Stroke                  0
Complication_risk       0
Overweight              0
Arthritis               0
Diabetes                0
Hyperlipidemia          0
BackPain                0
Anxiety                 0
Allergic_rhinitis       0
Reflux_esophagitis      0
Asthma                  0
Services                0
Initial_days            0
TotalCharge             0
Additional_charges      0
dtype: int64
```

In [10]:

```
#Checking for outliers for Population
sns.boxplot(x = 'Population', data = df)
```

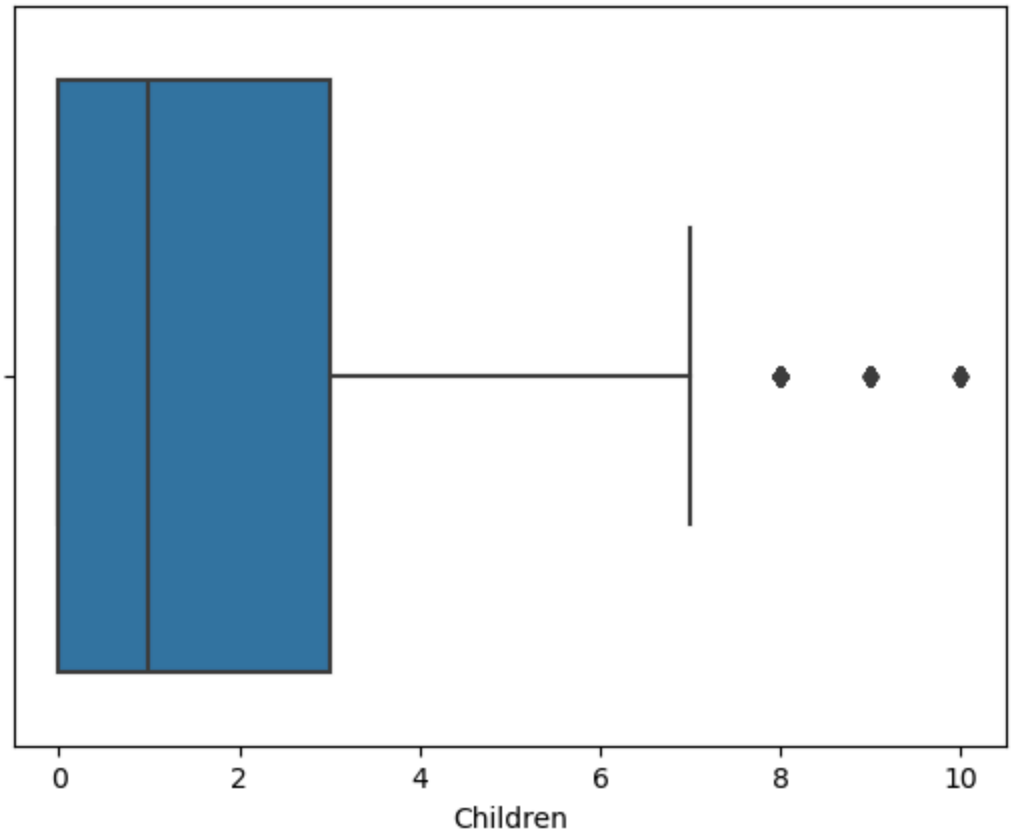
Out[10]:

```
<Axes: xlabel='Population'>
```



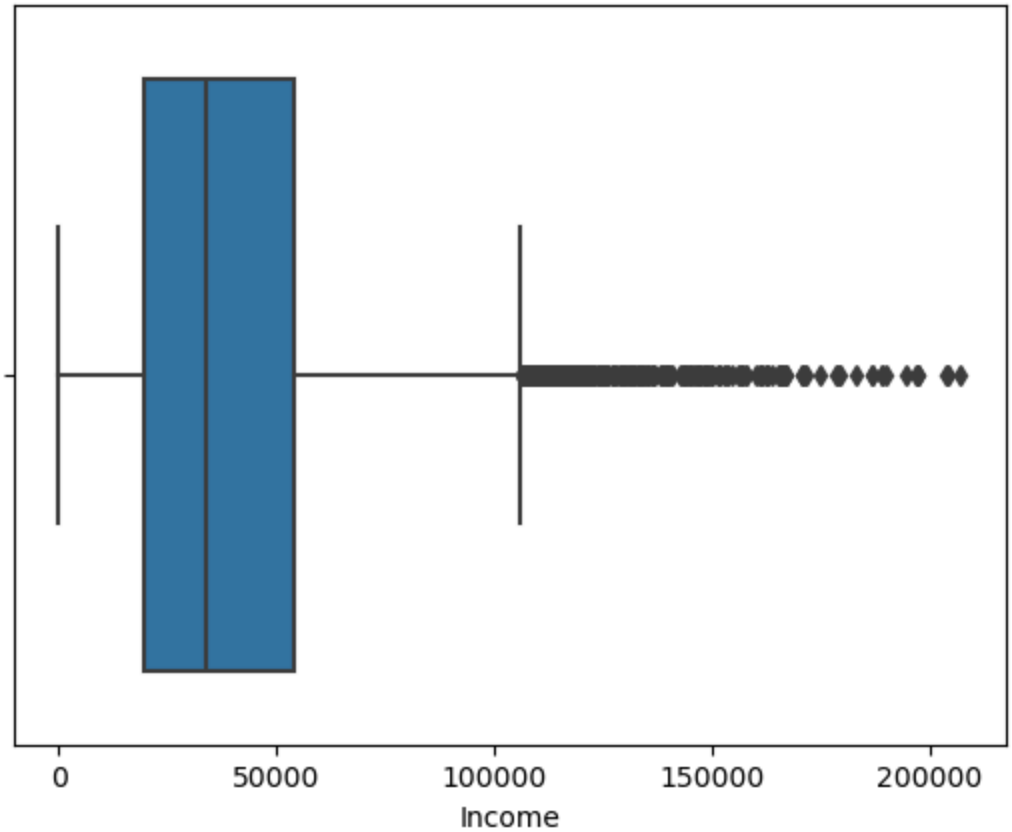
In [11]:
```
#Checking for outliers for Children
sns.boxplot(x = 'Children', data = df)
```

Out[11]: <Axes: xlabel='Children'>



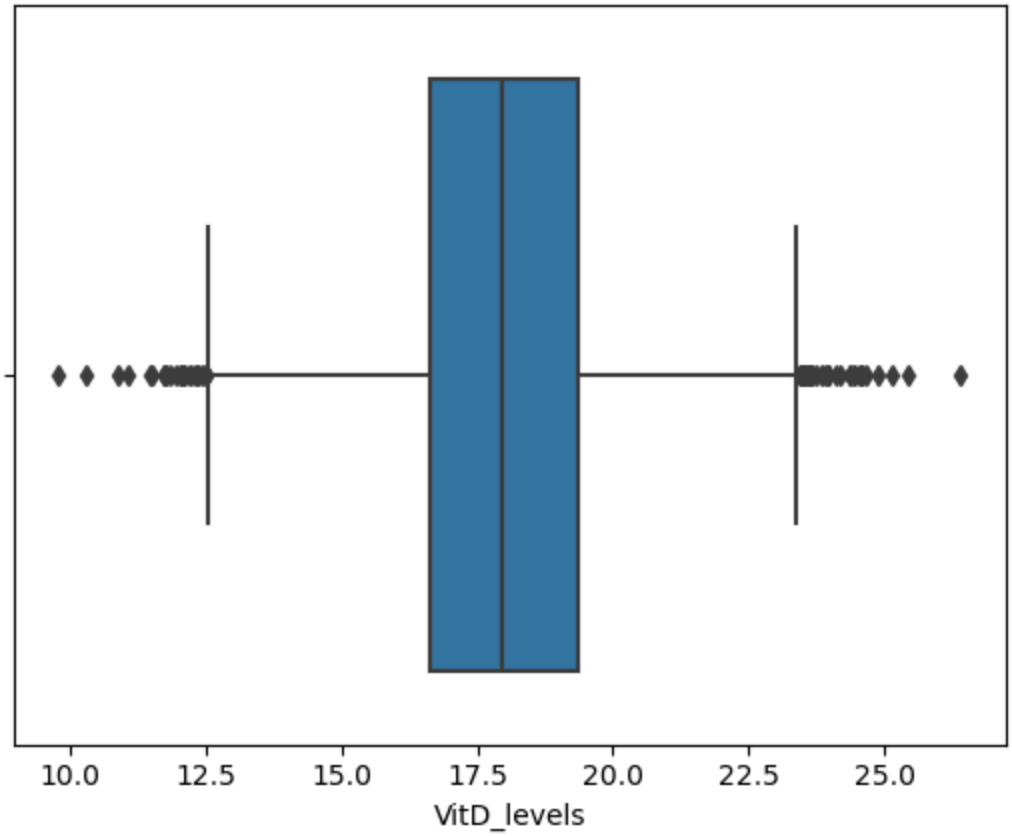
In [12]:
```
#Checking for outliers for Income
sns.boxplot(x = 'Income', data = df)
```

Out[12]: <Axes: xlabel='Income'>



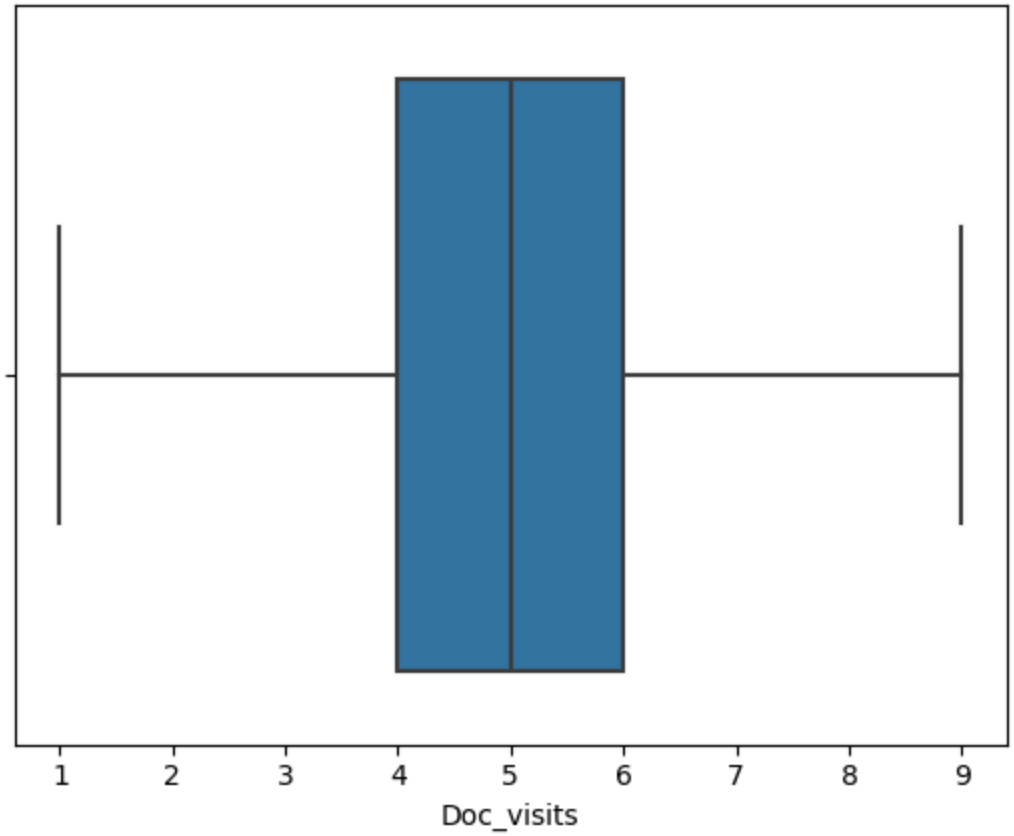
In [13]:
```
#Checking for outliers for VitD_levels
sns.boxplot(x = 'VitD_levels', data = df)
```

Out[13]: <Axes: xlabel='VitD_levels'>



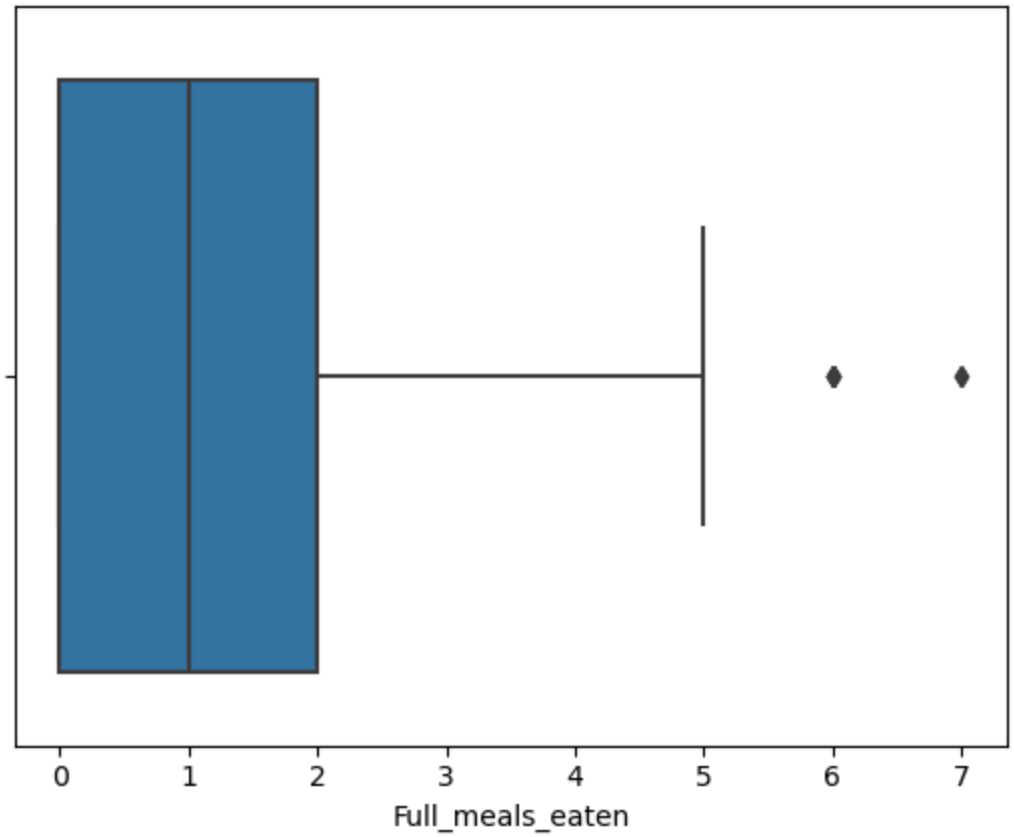
In [14]:
```
#Checking for outliers for Doc_visits
sns.boxplot(x = 'Doc_visits', data = df)
```

Out[14]:
```
<Axes: xlabel='Doc_visits'>
```



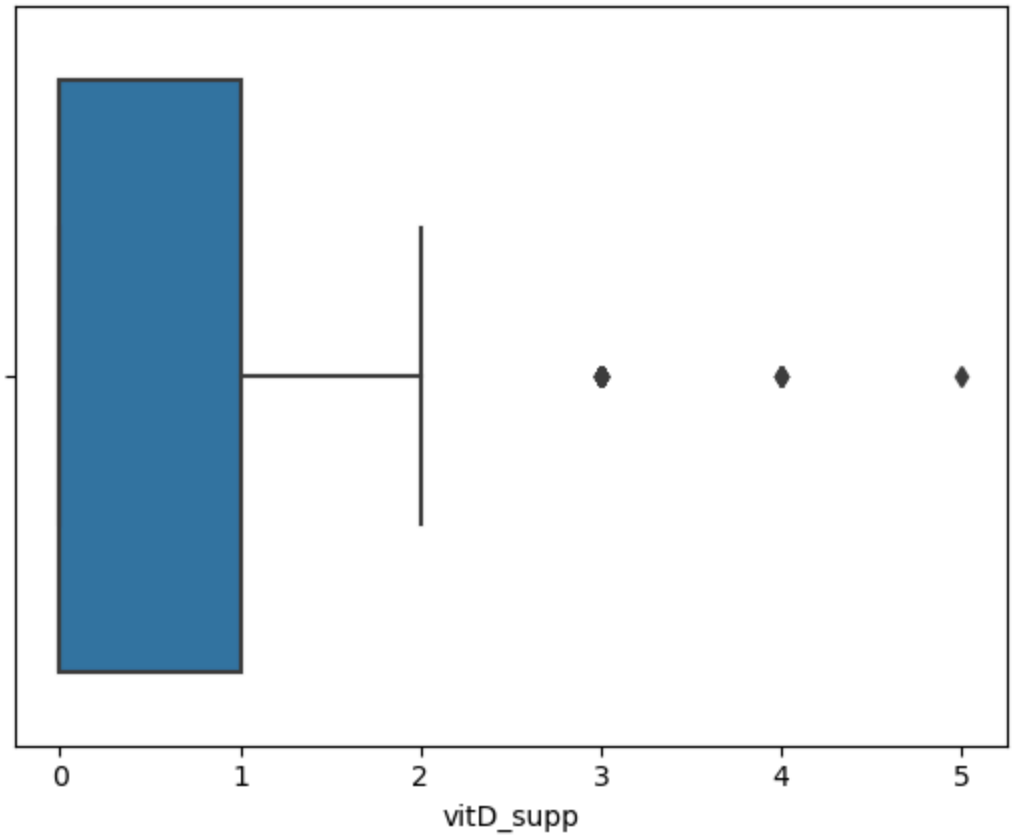
In [15]:
```
#Checking for outliers for Full meals eaten
sns.boxplot(x = 'Full_meals_eaten', data = df)
```

Out[15]:
```
<Axes: xlabel='Full_meals_eaten'>
```



In [16]:
```
#Checking for outliers for Vitamin D supplement
sns.boxplot(x = 'vitD_supp', data = df)
```

Out[16]: <Axes: xlabel='vitD_supp'>



In [17]:
```
#Feature scaling for numerical columns, excluding the target variable 'ReAdmis'
scaler = StandardScaler()
```

```
numerical_cols = df.select_dtypes(include=['float64', 'int64']).columns.difference(['ReAdmis'])
df[numerical_cols] = scaler.fit_transform(df[numerical_cols])
```

In [18]:
```
#Split the Data into Features (X) and Target (y)
X = df.drop(columns=['ReAdmis'])
y = df['ReAdmis']
```

In [19]:
```
#Split the Data into Training and Test Sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

In [20]:
```
#Save training and testing data to CSV files
X_train.to_csv("X_train.csv", index=False)
X_test.to_csv("X_test.csv", index=False)
y_train.to_csv("y_train.csv", index=False)
y_test.to_csv("y_test.csv", index=False)
```

In [21]:
```
#Load the saved files to check if they were saved correctly
X_train_loaded = pd.read_csv("X_train.csv")
X_test_loaded = pd.read_csv("X_test.csv")
y_train_loaded = pd.read_csv("y_train.csv")
y_test_loaded = pd.read_csv("y_test.csv")
```

In [22]:
```
#The KNN Model
knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train, y_train)

#Prediction
y_pred_proba= knn_model.predict_proba(X_test)[:,1]
y_pred = (y_pred_proba > 0.5).astype(int)
```

In [23]:
```
#Accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy}")
```

```
Accuracy: 0.578
```

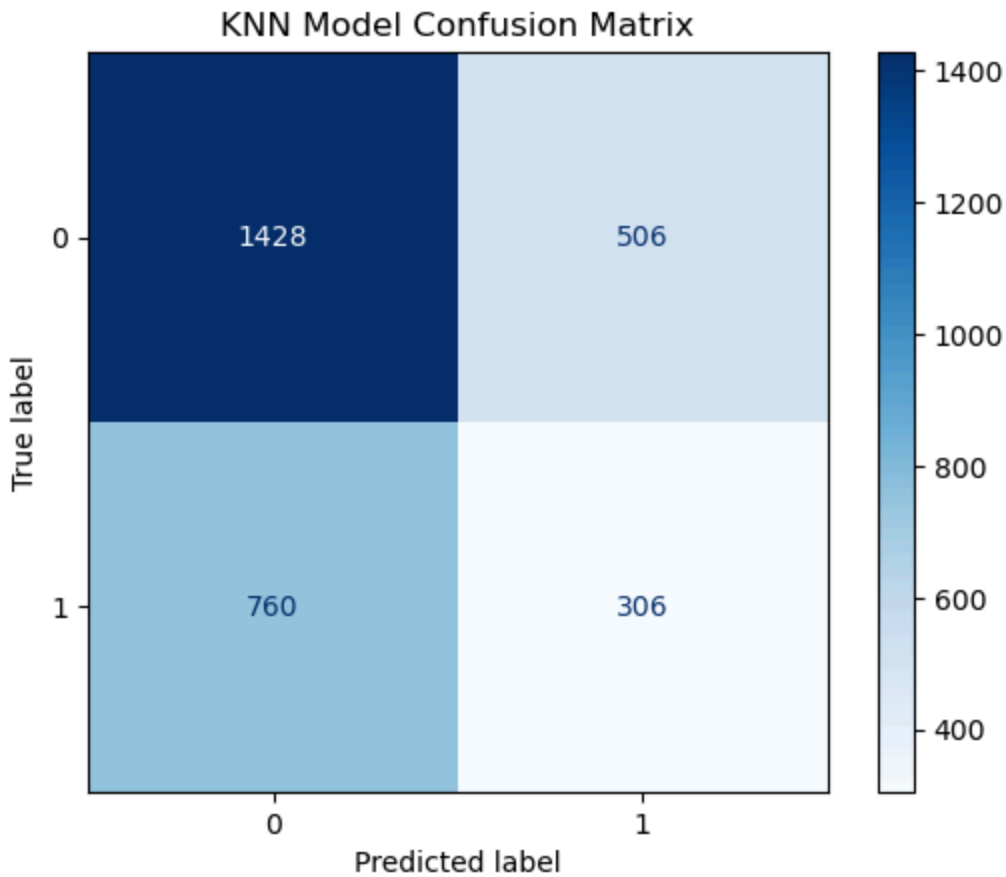
In [24]:
```
#Confusion Matrix
conf_matrix_knn = confusion_matrix(y_test, y_pred)
print(f"Confusion Matrix: \n{conf_matrix_knn}")
```

```
Confusion Matrix:
[[1428  506]
 [ 760  306]]
```

In [25]:
```
#AUC (Area Under the Curve)
auc_knn = roc_auc_score(y_test, y_pred)
print(f"AUC: {auc_knn}")
```

```
AUC: 0.5127102448337346
```

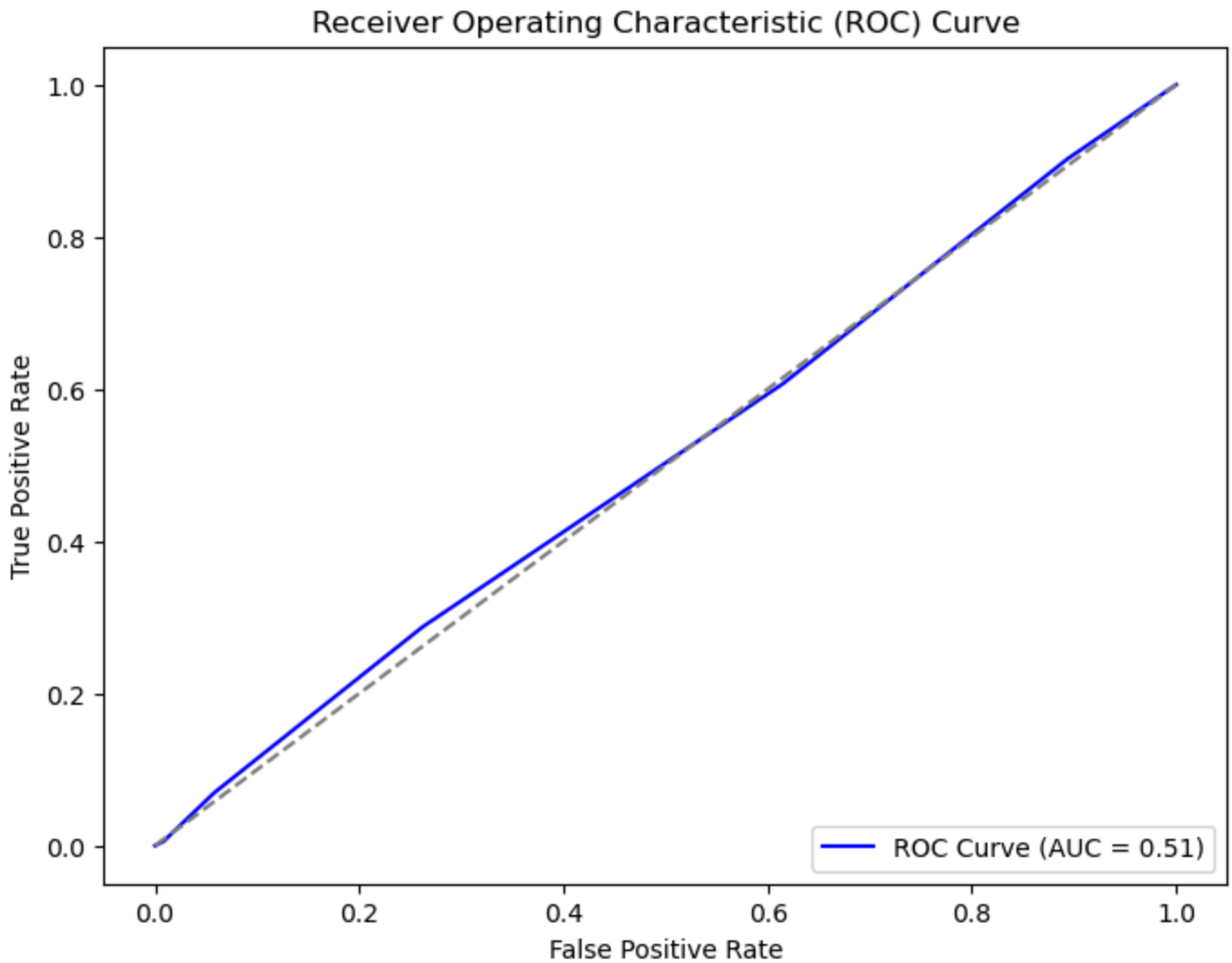
In [26]:
```
#Visualizing the Confusion Matrix
disp = ConfusionMatrixDisplay(confusion_matrix=conf_matrix_knn)
disp.plot(cmap=plt.cm.Blues)
plt.title('KNN Model Confusion Matrix')
plt.show()
```



In [27]:

```
#ROC Curve
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)

plt.figure(figsize=(8,6))
plt.plot(fpr, tpr, color='blue', label='ROC Curve (AUC = %0.2f)' % auc_knn)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc='lower right')
plt.show()
```



In [28]:
```
#Save cleaned dataset
new_data_path = 'output/new_medical_data_D209_I.csv'
df.to_csv(new_data_path, index=False)
```

In [ ]: